Documentação de Arquitetura

1. Visão geral

O Sistema de Recursos Humanos é uma aplicação web com frontend e backend separados, organizada como um monólito modular. A solução atende dois grandes domínios:

- **Operação de RH**: admissão, demissão, aprovações, checklists, departamentos, cargos e controle de acesso.
- **Pesquisas internas**: pesquisas, versões, dimensões, perguntas, campanhas, público-alvo, respostas e KPIs.

A arquitetura atual favorece simplicidade operacional, desenvolvimento rápido e baixo atrito para evolução do MVP.

```
Usuário
  -> Frontend React
  -> API REST FastAPI
  -> SQLAlchemy ORM
  -> SQLite em desenvolvimento local
  -> PostgreSQL no deploy
```

2. Tecnologias principais

Frontend

- React;
- Vite;
- React Router;
- CSS global próprio;
- chamadas HTTP via serviços internos baseados em `fetch`.

Backend

- FastAPI;
- SQLAlchemy ORM;
- Pydantic;
- JWT para sessão;
- autenticação local e integração opcional com LDAP/Active Directory;
- Alembic disponível para migrations.

Banco de dados

- SQLite como padrão de desenvolvimento local;
- PostgreSQL 16 no deploy via Docker Swarm;
- modelos ORM centralizados em `backend/app/models`;
- migrations Alembic aplicadas no startup do backend em produção;
- seed inicial para ambiente local.

Deploy

- Dockerfile na raiz;
- `docker-compose.yml`;
- Nginx para servir frontend em container;
- PostgreSQL 16 em container com volume persistente;
- GitHub Actions para deploy em produção;
- publicação em Docker Swarm com Traefik.

3. Organização do repositório

```
backend/
  app/
    api/
      deps.py
      v1/
        router.py
        endpoints/
          access_control.py
          admin.py
          auth.py
          health.py
          public.py
    core/
      config.py
      security.py
    db/
      base.py
      session.py
    models/
    schemas/
    seeds/
    services/
    main.py
  alembic/
  requirements.txt
  seed.py

frontend/
  public/
  src/
    auth/
    components/
    pages/
    services/
    styles/
    utils/
    App.jsx
    main.jsx
  package.json
  vite.config.js

Dockerfile
docker-compose.yml
README.md
ARCHITECTURE.md
```

4. Camadas do frontend

`frontend/src/App.jsx`

Responsável pelo roteamento principal da aplicação.

Define rotas públicas, rotas protegidas, portal administrativo e restrições por módulo.

Principais grupos de rotas:

- home;
- solicitações;
- pesquisas públicas;
- campanha pública;
- login administrativo;
- portal administrativo.

`frontend/src/auth/AuthProvider.jsx`

Gerencia a sessão do usuário:

- login;
- logout;
- token JWT;
- carregamento do usuário atual;
- estado autenticado para rotas protegidas.

`frontend/src/components`

Componentes reutilizáveis:

- layout administrativo;
- rotas protegidas;
- modais de detalhes;
- modais de checklist;
- modais de status de aprovação;
- modais de contratação;
- blocos de listagem de solicitações.

`frontend/src/pages`

Telas da aplicação, geralmente alinhadas a um caso de uso:

- páginas públicas de home, solicitações e pesquisas;
- formulários de admissão e demissão;
- minhas solicitações;
- páginas administrativas;
- dashboards;
- cadastros;
- gestão de pesquisas;
- análise de campanhas.

`frontend/src/services`

Serviços de comunicação com o backend:

- autenticação;
- endpoints administrativos;
- endpoints públicos.

`frontend/src/utils`

Utilitários de apoio, como:

- controle de acesso por módulo;
- leitura de status de campanhas.

5. Camadas do backend

`backend/app/main.py`

Ponto de entrada da API.

Responsabilidades:

- criar a aplicação FastAPI;
- configurar CORS;
- executar rotinas de inicialização compatíveis com o ambiente;
- sincronizar usuários LDAP no startup quando configurado;
- registrar o roteador versionado.

`backend/app/api/v1/router.py`

Centraliza o registro dos endpoints da versão 1 da API.

`backend/app/api/v1/endpoints/auth.py`

Responsável por:

- login;
- emissão de JWT;
- autenticação local;
- autenticação LDAP quando habilitada;
- endpoint de sessão atual.

`backend/app/api/v1/endpoints/admin.py`

Principal módulo administrativo.

Concentra endpoints de:

- dashboard;
- departamentos;
- cargos;
- solicitações de admissão;
- solicitações de demissão;
- filas de aprovação;
- checklists;
- pesquisas;
- dimensões;
- perguntas;
- publicação de campanhas;
- respostas de campanhas.

`backend/app/api/v1/endpoints/public.py`

Responsável pelas telas públicas de pesquisas.

Oferece endpoints para:

- listar campanhas publicadas;
- consultar detalhes de campanha;
- iniciar participação;
- enviar respostas.

`backend/app/api/v1/endpoints/access_control.py`

Responsável por controle administrativo de usuários sincronizados do diretório.

Permite:

- listar usuários LDAP ativos;
- atualizar papel de acesso.

`backend/app/api/deps.py`

Define dependências compartilhadas de API, principalmente:

- resolução do usuário autenticado;
- validação de acesso ao portal;
- validação de acesso administrativo.

`backend/app/core`

Responsável por configuração e segurança:

- variáveis de ambiente;
- CORS;
- JWT;
- hash de senha;
- papéis administrativos.

`backend/app/db`

Responsável pela persistência:

- engine;
- sessão;
- base declarativa;
- criação de tabelas;
- compatibilidade de schema.

`backend/app/models`

Entidades ORM do domínio.

Principais grupos:

- usuários e colaboradores;
- departamentos e cargos;
- pesquisas, versões, dimensões e perguntas;
- campanhas e audiência;
- respostas;
- solicitações de admissão;
- solicitações de desligamento;
- aprovações;
- checklists;
- auditoria.

`backend/app/schemas`

Contratos Pydantic de entrada e saída da API.

Os schemas separam o formato HTTP dos modelos ORM.

`backend/app/services`

Serviços reutilizáveis:

- autenticação LDAP;
- sincronização de usuários do diretório;
- definição de checklists padrão;
- regras auxiliares de acesso.

6. Modelo de segurança

Autenticação

O sistema suporta:

- autenticação local com senha armazenada no banco;
- autenticação LDAP/Active Directory para perfis de RH quando habilitada.

Mesmo com LDAP, o usuário precisa existir no banco para manter:

- papel;
- status ativo/inativo;
- trilha de auditoria;
- regras internas de autorização.

Sessão

Após login válido, o backend emite um JWT. O frontend usa esse token nas chamadas protegidas.

Fluxo:

```
POST /api/v1/auth/login
  -> valida credenciais
  -> emite access token
  -> frontend armazena token
  -> GET /api/v1/auth/me carrega usuário atual
```

Autorização por perfil

Perfis existentes:

- `RH_ADMIN`;
- `RH_ANALISTA`;
- `RH_PESQUISAS`;
- `GESTOR`;
- `DIRETOR_RAVI`;
- `COLABORADOR`.

Módulos de acesso:

- `DASHBOARD`;
- `ADMISSION`;
- `DISMISSAL`;
- `SURVEYS`;
- `APPROVALS`;
- `ACCESS_CONTROL`.

Regras principais:

- `RH_ADMIN` possui acesso amplo;
- `RH_PESQUISAS` acessa pesquisas;
- `RH_ANALISTA` acessa admissão e demissão;
- `GESTOR` e `DIRETOR_RAVI` acessam aprovações;
- `COLABORADOR` não acessa o portal administrativo.

7. Rotas do frontend

Rotas públicas e colaborativas

```
/                                Home
/solicitacoes                    Hub de solicitações
/solicitacoes/admissao           Formulário de admissão
/solicitacoes/demissao           Formulário de demissão
/my-requests                     Minhas solicitações
/pesquisas                       Lista pública de campanhas
/campaigns/:campaignId           Resposta de campanha
/campaigns/:campaignId/thank-you Agradecimento
```

Rotas administrativas

```
/admin                          Início administrativo
/admin/login                    Login
/admin/dashboard                Dashboard geral
/admin/dashboard/pesquisas      Dashboard de pesquisas
/admin/dashboard/admissao       Dashboard de admissão
/admin/requests                 Solicitações administrativas
/admin/approvals                Aprovações
/admin/departments              Departamentos
/admin/job-titles               Cargos
/admin/admission-requests       Solicitações de admissão
/admin/admission-checklist      Checklist de admissão
/admin/dismissal-requests       Solicitações de demissão
/admin/dismissal-checklist      Checklist de demissão
/admin/surveys                  Pesquisas
/admin/surveys/:surveyId        Detalhe de pesquisa
/admin/campaigns/:id/responses  Respostas da campanha
/admin/campaigns/:id/kpis       KPIs da campanha
/admin/access-control           Controle de acesso
```

8. API HTTP

Prefixo versionado:

```
/api/v1
```

Saúde

```
GET /api/v1/health
```

Autenticação

```
POST /api/v1/auth/login
GET  /api/v1/auth/me
```

Público de pesquisas

```
GET  /api/v1/campaigns/published
GET  /api/v1/campaigns/published/{campaign_id}
POST /api/v1/campaigns/published/{campaign_id}/start
POST /api/v1/campaigns/published/{campaign_id}/submit
```

Administração

Principais grupos:

```
GET    /api/v1/admin/dashboard

GET    /api/v1/admin/departments
POST   /api/v1/admin/departments
PATCH  /api/v1/admin/departments/{department_id}

GET    /api/v1/admin/job-titles
POST   /api/v1/admin/job-titles
PATCH  /api/v1/admin/job-titles/{job_title_id}

GET    /api/v1/admin/hr/my-requests
GET    /api/v1/admin/hr/recruiters

GET    /api/v1/admin/hr/approvals/admission
POST   /api/v1/admin/hr/approvals/admission/{request_id}/approve
POST   /api/v1/admin/hr/approvals/admission/{request_id}/reject

GET    /api/v1/admin/hr/approvals/dismissal
POST   /api/v1/admin/hr/approvals/dismissal/{request_id}/approve
POST   /api/v1/admin/hr/approvals/dismissal/{request_id}/reject

GET    /api/v1/admin/hr/admission-requests
GET    /api/v1/admin/hr/admission-requests/{request_id}
POST   /api/v1/admin/hr/admission-requests
POST   /api/v1/admin/hr/admission-requests/{request_id}/checklist-progress
POST   /api/v1/admin/hr/admission-requests/{request_id}/hire
POST   /api/v1/admin/hr/admission-requests/{request_id}/finalize
GET    /api/v1/admin/hr/admission-requests/{request_id}/approval-status

GET    /api/v1/admin/hr/dismissal-requests
GET    /api/v1/admin/hr/dismissal-requests/{request_id}
POST   /api/v1/admin/hr/dismissal-requests
POST   /api/v1/admin/hr/dismissal-requests/{request_id}/checklist-progress
POST   /api/v1/admin/hr/dismissal-requests/{request_id}/reject
GET    /api/v1/admin/hr/dismissal-requests/{request_id}/approval-status
```

Checklists

```
GET    /api/v1/admin/admission-checklist
POST   /api/v1/admin/admission-checklist
PUT    /api/v1/admin/admission-checklist/{step_id}
DELETE /api/v1/admin/admission-checklist/{step_id}
POST   /api/v1/admin/admission-checklist/reorder
POST   /api/v1/admin/admission-checklist/reset-default

GET    /api/v1/admin/dismissal-checklist
POST   /api/v1/admin/dismissal-checklist
PUT    /api/v1/admin/dismissal-checklist/{step_id}
DELETE /api/v1/admin/dismissal-checklist/{step_id}
POST   /api/v1/admin/dismissal-checklist/reorder
POST   /api/v1/admin/dismissal-checklist/reset-default
```

Pesquisas

```
GET    /api/v1/admin/surveys
POST   /api/v1/admin/surveys
GET    /api/v1/admin/surveys/{survey_id}
PUT    /api/v1/admin/surveys/{survey_id}
DELETE /api/v1/admin/surveys/{survey_id}

POST   /api/v1/admin/surveys/{survey_id}/dimensions
PATCH  /api/v1/admin/dimensions/{dimension_id}
DELETE /api/v1/admin/dimensions/{dimension_id}

POST   /api/v1/admin/surveys/{survey_id}/questions
PATCH  /api/v1/admin/questions/{question_id}
DELETE /api/v1/admin/questions/{question_id}

POST   /api/v1/admin/surveys/{survey_id}/publish
GET    /api/v1/admin/campaigns/{campaign_id}/responses
```

Controle de acesso

```
GET /api/v1/admin/access-control/users
PUT /api/v1/admin/access-control/users/{user_id}
```

9. Domínio de admissão

Entidades principais

- `AdmissionRequest`;
- `AdmissionRequestApproval`;
- `AdmissionRequestCandidate`;
- `AdmissionChecklistStep`;
- `ApprovalWorkflowTemplate`;
- `ApprovalWorkflowStep`;
- `Employee`.

Estados

```
PENDING
APPROVED
FINALIZED
REJECTED
```

Fluxo

```
Solicitante cria requisição
  -> sistema cria etapas de aprovação
  -> aprovadores decidem em ordem
  -> RH acompanha solicitação aprovada
  -> RH registra candidatos contratados
  -> sistema cria colaboradores quando aplicável
  -> RH finaliza solicitação
```

SLA

O SLA de admissão é contado a partir de um marco de aprovação específico do fluxo, associado à aprovação do gestor de RH.

10. Domínio de demissão

Entidades principais

- `DismissalRequest`;
- `DismissalRequestApproval`;
- `DismissalChecklistStep`;
- `ApprovalWorkflowTemplate`;
- `ApprovalWorkflowStep`.

Estados

```
PENDING
UNDER_REVIEW
APPROVED
REJECTED
CANCELED
```

Fluxo

```
Solicitante cria pedido
  -> sistema cria etapas de aprovação
  -> aprovadores decidem em ordem
  -> RH acompanha checklist
  -> RH pode rejeitar operacionalmente após aprovação quando necessário
```

11. Domínio de pesquisas

Entidades principais

- `Survey`;
- `SurveyVersion`;
- `SurveyDimension`;
- `SurveyQuestion`;
- `QuestionOption`;
- `Campaign`;
- `CampaignAudience`;
- `Response`;
- `ResponseItem`.

Estados de versão

```
DRAFT
PUBLISHED
ARCHIVED
```

Estados de campanha

```
DRAFT
ACTIVE
CLOSED
```

Estados de resposta

```
DRAFT
SUBMITTED
```

Fluxo

```
Admin cria pesquisa
  -> cria versão inicial
  -> cadastra dimensões
  -> cadastra perguntas
  -> publica campanha
  -> sistema congela público-alvo
  -> participante responde
  -> admin acompanha respostas e KPIs
```

12. Auditoria

O sistema possui entidade de auditoria para registrar ações relevantes.

Ações mapeadas:

- `CREATE`;
- `UPDATE`;
- `DELETE`;
- `LOGIN`;
- `PUBLISH`;
- `SUBMIT`.

Exemplos de eventos auditados:

- criação e alteração de pesquisas;
- publicação de campanha;
- envio de resposta;
- operações administrativas relevantes;
- login.

13. Persistência e schema

O banco local padrão é SQLite, configurado por `DATABASE_URL` com valor padrão `sqlite:///./rh_surveys.db`.

No deploy, o `docker-compose.yml` sobe um serviço PostgreSQL 16 e configura o backend com:

```
DATABASE_URL=postgresql+psycopg2://postgres:postgres@postgres:5432/sistema_de_pesquisas
```

Antes de iniciar a API em produção, o container do backend executa:

```
python -m alembic upgrade head
```

Assim, o schema do banco de produção é preparado pelas migrations Alembic antes do Uvicorn subir.

Pontos importantes:

- os modelos ORM são a referência principal do domínio;
- schemas Pydantic são a referência dos contratos HTTP;
- migrations Alembic são o caminho de evolução do schema no deploy;
- as rotinas de compatibilidade executadas no startup ajudam no ambiente local e em bases legadas, mas não substituem migrations em produção.

14. Integração LDAP

O LDAP é opcional e controlado por variáveis de ambiente.

Quando habilitado:

- perfis de RH podem autenticar via diretório corporativo;
- usuários do diretório podem ser sincronizados para o banco;
- autorização continua sendo feita pelo papel salvo internamente.

Variáveis principais:

```
LDAP_ENABLED
LDAP_SERVER_URI
LDAP_USE_SSL
LDAP_START_TLS
LDAP_VALIDATE_CERTIFICATES
LDAP_BIND_DN
LDAP_BIND_PASSWORD
LDAP_USER_BASE_DN
LDAP_USER_FILTER
LDAP_USER_DN_TEMPLATE
LDAP_TIMEOUT_SECONDS
```

15. Configuração de ambiente

O backend centraliza configurações em `backend/app/core/config.py`.

Categorias de configuração:

- nome da aplicação;
- modo debug;
- prefixo da API;
- CORS;
- banco de dados via `DATABASE_URL`;
- JWT;
- LDAP.

O frontend usa Vite e pode consumir a API por URL configurada conforme o ambiente.

16. Deploy e infraestrutura

Desenvolvimento local

Backend:

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python seed.py
uvicorn app.main:app --reload
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Produção

O fluxo documentado no projeto usa:

- GitHub Actions;
- SSH para servidor;
- Docker;
- Docker Swarm;
- PostgreSQL 16 com volume `postgres-data`;
- Alembic executado automaticamente antes da API;
- Traefik;
- domínio `systemrh.wasion.com.br`.

17. Decisões arquiteturais

Monólito modular

Motivo:

- simplifica deploy;
- reduz custo operacional;
- facilita evolução inicial com equipe pequena.

Trade-off:

- maior concentração de regras no backend principal;
- necessidade futura de separar melhor casos de uso críticos.

FastAPI

Motivo:

- produtividade;
- tipagem;
- validação de payloads;
- documentação automática;
- bom encaixe com SQLAlchemy.

Trade-off:

- endpoints administrativos cresceram bastante e podem exigir refatoração por domínio.

React + Vite

Motivo:

- desenvolvimento rápido;
- boa experiência para SPA;
- roteamento simples.

Trade-off:

- páginas grandes podem acumular estado e regra de apresentação.

SQLite local e PostgreSQL no deploy

Motivo:

- simplicidade local;
- baixa fricção para MVP;
- fácil seed e depuração.
- banco PostgreSQL provisionado junto com a stack de produção;
- migrations Alembic aplicadas automaticamente no deploy.

Trade-off:

- o ambiente local continua simples, mas produção depende da saúde do serviço PostgreSQL e do volume persistente;
- mudanças de schema devem ser refletidas em migrations para manter o deploy reproduzível.

JWT

Motivo:

- sessão stateless;
- integração simples com SPA.

Trade-off:

- exige cuidado no armazenamento, expiração e invalidação de tokens.

18. Pontos fortes atuais

- separação clara entre frontend e backend;
- domínio funcional amplo já modelado;
- autorização por perfil e módulo;
- integração LDAP opcional;
- trilha de auditoria;
- checklists configuráveis;
- fluxos de aprovação reutilizados para admissão e demissão;
- campanhas de pesquisa com publicação e acompanhamento.

19. Pontos de atenção

- `admin.py` concentra muitas responsabilidades;
- parte da lógica de negócio está próxima dos endpoints;
- rotinas automáticas de compatibilidade ainda existem no startup;
- páginas administrativas podem crescer e precisar de decomposição;
- testes automatizados ainda devem ser ampliados para fluxos críticos;
- estratégia de sessão pode evoluir com refresh token, expiração mais refinada e políticas corporativas.

20. Recomendações de evolução

Prioridades sugeridas:

- extrair serviços de aplicação para admissão, demissão e pesquisas;
- dividir endpoints administrativos por domínio;
- ampliar cobertura de testes dos fluxos de aprovação;
- manter Alembic como caminho oficial de evolução de schema no deploy;
- reduzir gradualmente ajustes automáticos de schema no startup;
- fortalecer observabilidade com logs estruturados e métricas;
- documentar contratos de API com exemplos de payload;
- revisar política de expiração e renovação de sessão;
- revisar backup, retenção e restauração do volume PostgreSQL de produção.

21. Mapa rápido de responsabilidade

```
AuthProvider.jsx
  -> sessão frontend

ProtectedRoute.jsx
  -> proteção de rotas

accessControl.js
  -> regras de exibição por módulo no frontend

admin.py
  -> regras administrativas e operações principais

public.py
  -> participação pública em pesquisas

auth.py
  -> login, JWT e sessão

access_control.py
  -> gestão de usuários LDAP e papéis

models/
  -> entidades persistidas

schemas/
  -> contratos HTTP

services/
  -> integrações e regras auxiliares
```